

ContextGraph

Project Planner

A cross-AI personal context engine — development phases, timeline, and roadmap

Project Overview

Problem

Every AI session starts from zero — context is siloed per platform, non-portable, and flat, with no scope containment between clients or projects.

Solution

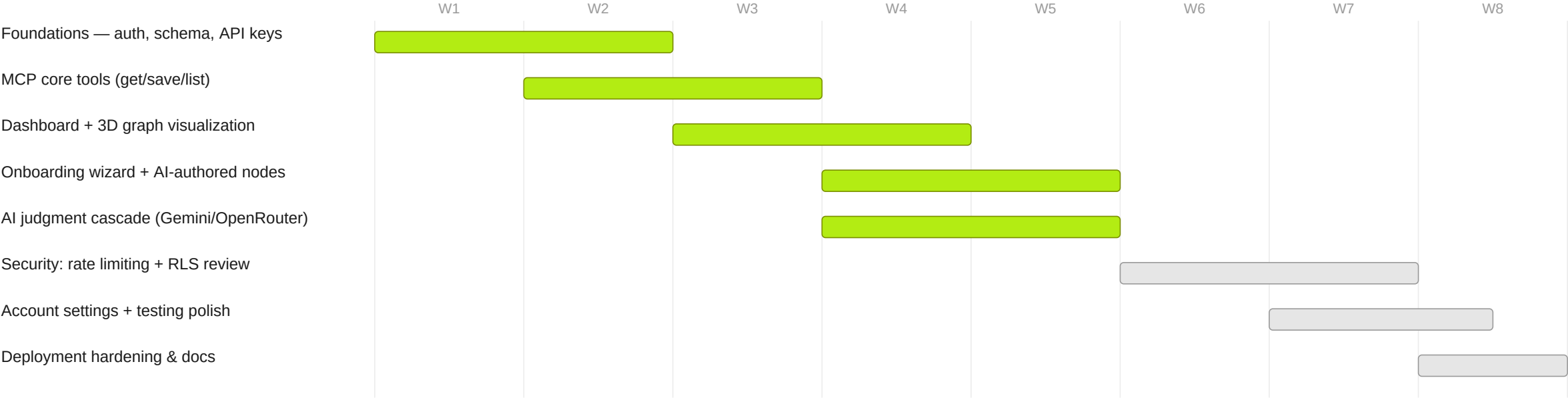
A structured, hierarchical context graph, owned by the user, reachable by any MCP-compatible AI via a personal API key, and visualized in a web dashboard.

Primary users	Secondary users
Developers / AI power users Multiple tools, multiple projects	Freelancers / agency owners Context per client

Development Phases

Phase 1	Phase 2	Phase 3
Done Personal MCP server — TypeScript + Node.js, GitHub-markdown backed. Two tools: get_context, save_context. Connected to Claude.ai, Claude Desktop, Claude Code.	Skipped Moved directly to Phase 3.	Current Multi-user SaaS — Supabase-backed graph, Better Auth accounts, web dashboard, per-user API keys.

Timeline & Milestones



Done Pending

Task Breakdown (V1)

Module	Status	Notes
Auth (email/password + Google)	Done	Better Auth, proxy.ts route guard
Onboarding wizard	Done	Structured 5-step form, AI-authored node content
Dashboard graph view	Done	react-force-graph-3d, realtime updates
MCP endpoint (3 tools)	Done	JSON-RPC 2.0, API-key auth
Weekly relevance decay	Done	Cron route
Account settings (edit/delete)	Pending	UI exists, not wired to a route
MCP rate limiting	Pending	Table exists, not enforced
RLS / anon-key review	Pending	Needs verification against live Supabase project

Risks & Mitigations

- **No enforced rate limiting on the MCP endpoint**

Mitigation: Wire up the existing rate_limits table before opening the endpoint to more clients.

- **Tenant isolation is fully programmatic (no RLS)**

Mitigation: Audit every lib/db.ts call for a userId filter; verify anon-key realtime exposure on the live project.

- **Two parallel hierarchy fields (parent_scope + context_edges) can drift**

Mitigation: Update both on every write, or collapse to a single representation.

- **AI judgment depends on third-party model availability**

Mitigation: Existing Gemini-first / OpenRouter fallback cascade already retries across models on 429/404/502/503.

Success Criteria

- 1 Sign up and complete onboarding.
- 2 See the context graph in the dashboard.
- 3 Connect any MCP-compatible AI using a personal API key.
- 4 Start a session where `get_context` loads context automatically.
- 5 Type `/save` and see the new entry appear in the dashboard graph.

Post-V1 Roadmap

- AI memory export parsing (ChatGPT/Claude memory → context graph) as a first-class import.
- Multiple API keys with labels and scoped permissions (e.g. a read-only key for a docs bot).
- Node version history.
- Public read-only graph sharing (e.g. share one project's context with a client).
- Browser extension that auto-injects active context into web-based chat UIs.